

SDACS Remote Flutter Access

Using Tailscale

Partner Setup Guide for Running the Flutter App Against the Raspberry Pi Backend

Project	SDACS - Smart Distributed Acoustic Control System
Audience	SDACS partners/team members installing on personal Windows PCs
Prepared for	Remote Flutter testing through the Raspberry Pi FastAPI backend
Version	1.0
Last updated	July 7, 2026

Core rule: use the Raspberry Pi Tailscale IP address, not Kyle's local 192.168.x.x LAN address, when working from home or another network.

Keep this guide with the team setup notes so a partner can reproduce the remote connection without opening port 8000 to the public internet.

Contents

1. Purpose and Audience
 2. System Overview
 3. Prerequisites
 4. Kyle/Admin Preparation
 5. Partner Installation Steps
 6. Verify the Tailscale Connection
 7. Test the Raspberry Pi Backend
 8. Run the Flutter App Remotely
 9. Confirm the App Is Using Tailscale
 10. Troubleshooting
 11. Security Rules
 12. Maintenance and Handoff
- Appendix A: Sources Checked

1. Purpose and Audience

This guide explains how a SDACS partner can run the Flutter app on their own Windows computer while using the Raspberry Pi backend remotely over Tailscale. The goal is to make remote development and GUI testing possible without putting the Raspberry Pi FastAPI service directly on the public internet.

Plain-language goal

Run Flutter locally on a partner computer, but point the app to the Raspberry Pi backend through the Raspberry Pi's private Tailscale address, which will look like 100.x.x.x.

When this guide applies

- The Raspberry Pi backend is already running on Kyle's SDACS server setup.
- A partner wants to run the Flutter app from home, campus, or another network.
- The partner has the Flutter repo cloned locally and can run Flutter commands.
- Kyle or the Tailscale admin can provide access to the SDACS Tailscale network or share the Raspberry Pi device.

When this guide does not apply

- If the partner is physically on Kyle's local network and intentionally testing the LAN address.
- If the Raspberry Pi backend is down or the FastAPI service is not listening on port 8000.
- If the Flutter repo has not been cloned yet or Flutter is not installed on the partner computer.

2. System Overview

Tailscale creates a private mesh network between approved devices. In this setup, the partner computer and Raspberry Pi both join the same private Tailscale network, called a tailnet, or the Raspberry Pi is shared with the partner's Tailscale account. The partner runs the Flutter UI locally, and the Flutter app sends API requests to the Raspberry Pi FastAPI backend through the Raspberry Pi Tailscale IP address.

Layer	Device or service	Role in this setup
Partner PC	Windows computer with Flutter	Runs the SDACS Flutter app locally.
Tailscale	Private 100.x.x.x network	Securely routes traffic between partner PC and Raspberry Pi without public port forwarding.
Raspberry Pi	SDACS backend server	Runs FastAPI on port 8000 and serves JSON data to Flutter.
SDACS stack	MQTT / Node-RED / data pipeline	Provides the backend with node telemetry and acoustic features.

Network path

Flutter app on partner PC -> Tailscale private tunnel -> Raspberry Pi Tailscale IP:8000 -> SDACS FastAPI backend -> latest node data.

Why the 100.x.x.x address matters

Tailscale assigns every device a stable Tailscale IP address in the 100.x.y.z range. This is the address partners should use remotely. A local LAN address like 192.168.5.61 is only reachable from the same local network unless additional routing is configured.

3. Prerequisites

Required item	Who provides it	Notes
Tailscale installed on partner PC	Partner	Use the official Windows installer. Windows 10 or later is expected for the current Windows client.
Access to the SDACS tailnet or shared Raspberry Pi	Kyle / Tailscale admin	Do not share a single Tailscale account. Each person should sign in with their own accepted invite/account.
Raspberry Pi Tailscale IP	Kyle	It will look like 100.x.x.x. Example placeholder: 100.101.102.103.
Flutter repo cloned locally	Partner	Recommended path in this guide: C:\<local_repo>\flutter_app.
Flutter installed and working	Partner	Run flutter doctor separately if Flutter itself is not ready.
Backend running on Raspberry Pi	Kyle	FastAPI should respond on port 8000 before partners try to connect.

Do not use this from home

Do not use `http://192.168.5.61:8000` from home. That address is Kyle's local network address and only works when the client is on the same LAN or has special routing. Use `http://100.x.x.x:8000` instead.

4. Kyle/Admin Preparation

Before a partner starts, Kyle or the Tailscale admin should confirm that the Raspberry Pi is reachable and that the backend is healthy. This avoids making the partner troubleshoot a server-side problem.

4.1 Confirm the backend works locally on the Raspberry Pi

Raspberry Pi local backend test

```
# Run on the Raspberry Pi
curl http://127.0.0.1:8000/api/nodes/node03/features/latest
```

Expected result: a JSON response. If this fails on the Raspberry Pi itself, fix the backend before asking partners to test remote access.

4.2 Confirm the Raspberry Pi is online in Tailscale

Raspberry Pi Tailscale checks

```
# Run on the Raspberry Pi
tailscale status
tailscale ip -4
```

Record the Raspberry Pi Tailscale IPv4 address and send it to the partner. It should look like 100.x.x.x.

4.3 Choose how to grant partner access

There are two common options:

Option	Best use	Important note
Invite the partner to the SDACS tailnet	Best when partners may need access to several SDACS devices or future devices.	By default, an invited user may access devices in the tailnet unless access controls restrict them.
Share only the Raspberry Pi device	Best when partners only need access to one fixed machine.	The shared device is available without exposing it to the public internet, and the partner does not get full tailnet access.

4.4 Send the partner a minimal handoff

- Tailscale invite link or instructions for the team account/invite flow.
<https://login.tailscale.com/admin/invite/n7pXCQh9Yv9LrizOkDKw11>
- Raspberry Pi Tailscale IP: 100.90.8.58
- The backend base URL format: `http://100.90.8.58:8000`.

The Flutter repo branch or pull instructions to use.

```
cd C:\<localrepo>\flutter_app
git branch
git checkout feature/flutter-api-backend
flutter pub get
flutter upgrade // If out of date
flutter run -d chrome --dart-define=SDACS_BACKEND_IP=100.90.8.58 --dart
define=SDACS_BACKEND_PORT=8000
```

- Test the backend using tailscale IP:
Invoke-RestMethod "http://100.90.8.58:8000/api/nodes" | ConvertTo-Json -Depth 10

5. Partner Installation Steps

5.1 Install Tailscale on Windows

1. Open the official Tailscale download page or the official Windows install documentation.
2. Download the latest Windows .exe installer.
<https://tailscale.com/download/windows>
3. Run the installer. After installation, look for the Tailscale icon in the Windows system tray. If it is hidden, click the up arrow in the tray overflow area.
4. Right-click the Tailscale icon to view status and account options.
5. Select Log in and authenticate with the team invite/account or the identity provider specified by Kyle.
6. Wait until Tailscale shows Connected.

5.2 Accept the SDACS access invitation

Open the invite link from Kyle in a browser.

(<https://login.tailscale.com/admin/invite/n7pXCQh9Yv9LrizOkDKw11>)

Sign in to Tailscale using your own account or the requested identity provider.

Accept the invitation or shared-device access prompt.

Return to the Tailscale desktop app and confirm it is connected to the expected tailnet or has the Raspberry Pi shared device visible.

Account rule

Do not share one Tailscale login across multiple people. Each partner should use their own Tailscale account/invite so access can be revoked or audited if needed.

6. Verify the Tailscale Connection

Open PowerShell on the partner computer and run the following checks.

Check that the partner PC is connected

```
tailscale status
tailscale ip -4
```

Expected results:

- The partner PC has its own 100.x.x.x address from `tailscale ip -4`.
- The Raspberry Pi appears in `tailscale status`, or at least the partner can reach the Raspberry Pi IP Kyle provided.
- Tailscale shows Connected in the tray app.

```
PS C:\Users\kyled> tailscale status
100.87.19.66   cleanndecent  kyle.david.dick@  windows  -
100.90.8.58    sdacs-broker   kyle.david.dick@  linux    -
PS C:\Users\kyled> tailscale ip -4
100.87.19.66
```

6.1 Ping the Raspberry Pi over Tailscale

Tailscale-only ping

```
$RPI_TS_IP = "100.90.8.58"
tailscale ping $RPI_TS_IP
```

Expected result: Tailscale reports successful pings to the Raspberry Pi. If this fails, the problem is Tailscale access, the wrong IP, the Raspberry Pi being offline, or an access-control issue.

```
PS C:\Users\kyled> $RPI_TS_IP = "100.90.8.58"
PS C:\Users\kyled> tailscale ping $RPI_TS_IP
pong from sdacs-broker (100.90.8.58) via 192.168.5.61:41641 in 11ms
```

7. Test the Raspberry Pi Backend

After Tailscale connectivity is confirmed, test the SDACS FastAPI backend directly from PowerShell. This separates backend connectivity from Flutter app issues.

PowerShell backend test

```
$RPI_TS_IP = "100.90.8.58"
$API_BASE = "http://{RPI_TS_IP}:8000"

Invoke-RestMethod "$API_BASE/api/nodes/node03/features/latest" | ConvertTo-Json -Depth 10
```

```

PS C:\Users\kyled> $RPI_TS_IP = "100.90.8.58"
PS C:\Users\kyled> $API_BASE = "http://{$RPI_TS_IP}:8000"
PS C:\Users\kyled>
PS C:\Users\kyled> Invoke-RestMethod "$API_BASE/api/nodes/node03/features/latest" | ConvertTo-Json -Depth 10
{
  "id": 18,
  "timestamp_iso": "99975168",
  "timestamp_us": 99975168,
  "node_id": "node03",
  "topic": "sdacs/node/node03/features",
  "record_type": "features",
  "fw_version": "ota-enable-v2",
  "status": null,
  "capture_state": null,
  "phase": null,
  "seq": 4,
  "n": 26624,
  "rms": 4.4E-05,
  "dbfs": -87.2,
  "db_spl": 32.8,
  "f_peak_hz": 46.9,
  "p2p_raw": 2855,
  "zeros": 30,
  "temp_c": 21.08,
  "rh_percent": 62.49,
  "batt_soc_percent": 0.0,
  "batt_voltage_v": 3.0638,
  "batt_charge_rate_pct_per_hr": 0.0,
  "batt_sample_count": null,
  "batt_error_count": null,
  "batt_valid": 1,
  "rssi_dbm": null,
  "free_heap": null,
  "wifi_connected": null,
  "mqtt_connected": null,
  "ota_ready": null,
  "ota_in_progress": null,
  "request_id": null,
  "cmd": null,
  "event": null,
  "reason": null,
  "raw_bytes": null,
  "wav_bytes": null,
  "metrics_bytes": null,
  "raw_path": null,
  "wav_path": null,
  "metrics_path": null,
  "ble_gateway": null,
  "ble_rssi_dbm": null,
  "ble_proximity": null,
  "ble_seen_count": null,
  "ble_address": null,
  "err": "1",
  "raw_json": "{\n  \"node_id\": \"node03\", \n  \"node\": \"node03\", \n  \"record_type\": \"features\", \n  \"timestamp\": 99975168, \n  \"fw_version\": \"ota-enable-v2\", \n  \"seq\": 4, \n  \"t_us\": 99975168, \n  \"n\": 26624, \n  \"rms\": 0.000044, \n  \"dbfs\": -87.2, \n  \"db_spl\": 32.8, \n  \"fft_peak_hz\": 46.9, \n  \"f_peak_hz\": 46.9, \n  \"fft_low_ratio\": 0.680361, \n  \"fft_mid_ratio\": 0.250202, \n  \"fft_high_ratio\": 0.069436, \n  \"fft_total_energy\": 0.001757367, \n  \"p2p_raw\": 2855, \n  \"zeros\": 30, \n  \"temp_c\": 21.08, \n  \"rh_percent\": 62.49, \n  \"batt_soc_percent\": 0, \n  \"batt_voltage_v\": 3.0638, \n  \"batt_charge_rate_pct_per_hr\": 0, \n  \"batt_valid\": true, \n  \"err\": 1\n}"
}

```

Expected JSON

A successful response should be JSON, not an HTML error page. The exact fields depend on the current backend branch, but typical useful fields include:

- node_id, such as node03.
- record_type, such as features.
- timestamp_iso or another timestamp field.
- audio metrics such as rms, dbfs, db_spl, f_peak_hz, band ratios, or low/mid/high energy values.
- environment or battery fields if the backend currently includes them.

Optional browser checks

Browser URLs to try

```

http://100.90.8.58:8000/api/nodes
http://100.90.8.58:8000/docs

```

8. Run the Flutter App Remotely

Once the API returns JSON, run the Flutter app with the backend base URL passed in as a Dart define. The backend base URL should be only the scheme, IP, and port. Do not include /api at the end.

Preferred Flutter command

```
cd C:\<local_repo>\flutter_app
flutter pub get
$flutter run -d chrome --dart-define=SDACS_BACKEND_IP=100.90.8.58 --dart
define=SDACS_BACKEND_PORT=8000
```

9. Confirm the App Is Using Tailscale

Check	Good sign	Bad sign
Node list or dashboard cards	Nodes load without timeout errors.	The UI reports connection timeout or no backend response.
Latest feature values	Values update or display recent timestamps.	The app stays blank or shows stale demo/default data.
Backend logs on Raspberry Pi	GET requests appear for /api/nodes or feature endpoints.	No request appears when the app refreshes.
App URL/config logs	Base URL contains 100.x.x.x:8000.	Base URL contains 192.168.5.61, 192.168.5.40, localhost, or another stale IP.

Clean rebuild if stale

```
# If the app seems stuck on an old value
flutter clean
flutter pub get
$flutter run -d chrome --dart-define=SDACS_BACKEND_IP=100.90.8.58 --dart
define=SDACS_BACKEND_PORT=8000
```

10. Troubleshooting

Symptom	Most likely cause	Fix
Tailscale says Disconnected	The partner is not signed in or the local Tailscale service is not running.	Open Tailscale from the Start menu, sign in again, and confirm the tray icon says Connected.
`tailscale status` does not show the Raspberry Pi	Wrong tailnet, unaccepted invite, shared device not visible, or access controls are blocking visibility.	Confirm the partner accepted the correct invite. Kyle/admin should verify the user/device access in the Tailscale admin console.
`tailscale ping 100.x.x.x` fails	Wrong Raspberry Pi Tailscale IP, Raspberry Pi offline, Tailscale offline on the Pi, or ACL/sharing issue.	Kyle should run `tailscale status` and `tailscale ip -4` on the Pi, then resend the correct IP.
`Invoke-RestMethod` times out	Tailscale path may work, but FastAPI may not be reachable on port 8000.	Verify backend is running. Kyle should check server logs, port 8000, firewall rules, and whether the service is listening beyond 127.0.0.1.
`Invoke-RestMethod` says connection refused	The Raspberry Pi is reachable, but nothing is listening on port 8000.	Start or restart the FastAPI backend on the Raspberry Pi.
API test returns JSON, but Flutter still fails	Flutter is using the wrong environment variable, stale hardcoded IP, wrong branch, or cached build.	Try the fallback API_BASE_URL define, confirm `backend_config.dart`, and run `flutter clean`.
App still tries 192.168.5.61 or	The runtime define is not being applied	Run Flutter from the terminal with the

Symptom	Most likely cause	Fix
192.168.5.40	or the app has a hardcoded default overriding it.	full --dart-define command. Check launch.json or IDE run settings.
Works on Kyle's LAN but not from home	The app is using a local LAN IP instead of the Tailscale IP.	Use http://100.x.x.x:8000 for remote access.
Other VPN is connected	Another VPN may conflict with routes or DNS.	Disconnect the other VPN and retest Tailscale status, Tailscale ping, then the API.
Flutter cannot find a device	Flutter desktop/web target is not configured.	Run <code>`flutter devices`</code> , use <code>`-d windows`</code> or <code>`-d chrome`</code> , and fix Flutter setup separately if needed.

Backend bind warning

If FastAPI is only bound to 127.0.0.1 on the Raspberry Pi, remote clients will not reach it over Tailscale. For remote testing, the backend usually needs to listen on 0.0.0.0 or the Raspberry Pi's Tailscale interface/IP while firewall and Tailscale access rules restrict who can connect.

11. Security Rules

- **Do not expose port 8000 publicly.** Do not create router port forwards, public firewall openings, or public tunnels for the backend unless the team has explicitly designed and reviewed that deployment.
- **Use Tailscale access instead.** Remote partners should reach the Raspberry Pi through the private Tailscale IP or approved MagicDNS name.
- **Prefer least privilege.** If partners only need the Raspberry Pi, consider sharing only that device or using access controls to limit access to the required service.
- **Protect the Raspberry Pi IP and repo details.** The 100.x.x.x IP is not public internet exposure, but it should still be treated as internal project information.
- **Remove access when no longer needed.** If a partner leaves the project or a computer is lost, remove that device or revoke the invite from the Tailscale admin console.

12. Maintenance and Handoff

What Kyle should keep updated

- Current Raspberry Pi Tailscale IP and machine name.
- Current Flutter branch and expected backend environment variable name.
- Current backend port and endpoint used for health testing.
- Known working command for the latest Flutter branch.
- Any access-control restrictions that affect partner access.

What partners should report when something fails

- Screenshot of the Tailscale tray status.
- Output from ``tailscale status`` and ``tailscale ip -4``.
- Output or error from the PowerShell ``Invoke-RestMethod`` backend test.
- The exact ``flutter run`` command used and the first relevant error from the Flutter console.

Documentation note

This guide is intended to be easy to print or store with the SDACS project binder. Keep the final working IP/branch notes current so partners do not use stale backend addresses.

Appendix A - Sources Checked

These sources were checked while preparing this guide. The SDACS-specific commands and endpoint names come from the team-provided setup notes.

- Tailscale - Install Tailscale on Windows: <https://tailscale.com/docs/install/windows>
- Tailscale - What are these 100.x.y.z addresses?: <https://tailscale.com/docs/concepts/tailscale-ip-addresses>
- Tailscale - Inviting users vs sharing a device: <https://tailscale.com/docs/reference/inviting-vs-sharing>
- Tailscale - Invite any user to your tailnet: <https://tailscale.com/docs/features/sharing/how-to/invite-any-user>
- Tailscale - Tailscale CLI: <https://tailscale.com/docs/reference/tailscale-cli>
- Tailscale - MagicDNS: <https://tailscale.com/docs/features/magicdns>

Date accessed: July 7, 2026